

Proyecto Semana 05: Sistema de Configuración UI

Objetivo

Construir un sistema de configuración de interfaz de usuario utilizando composición de componentes, Context API y compound components para crear una experiencia de usuario rica y configurable.

Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio]

Adapta el proyecto al dominio asignado. Ejemplos:

- **Biblioteca:** Configurador de vista de catálogo
- **Farmacia:** Panel de preferencias de inventario
- **Gimnasio:** Configurador de dashboard de entrenamiento
- **Restaurante:** Panel de configuración de menú digital

Descripción

Crearás una aplicación con un panel de configuración que permite al usuario personalizar:

1. **Tema visual** (claro/oscuro/sistema)
2. **Tamaño de texto** (pequeño/mediano/grande)
3. **Densidad de contenido** (compacto/normal/espacioso)
4. **Preferencias de notificaciones**

La aplicación debe demostrar:

- Context API para estado global de configuración
- Compound Components para formularios y cards
- Composición con children y slots
- Persistencia de preferencias en localStorage

Tiempo Estimado

2-2.5 horas

Requisitos Funcionales

1. ConfigProvider (Context)

```
interface ConfigState {
  theme: 'light' | 'dark' | 'system';
  fontSize: 'small' | 'medium' | 'large';
  density: 'compact' | 'normal' | 'spacious';
  notifications: {
    email: boolean;
    push: boolean;
    sound: boolean;
  };
};
```

- Persistir en localStorage
- Detectar preferencia del sistema para tema
- Hook `useConfig` con validación

2. Compound Components

Card Compound Component:

```
<Card>
  <Card.Header>
    <Card.Title>Título</Card.Title>
    <Card.Actions>...</Card.Actions>
  </Card.Header>
  <Card.Body>Contenido</Card.Body>
  <Card.Footer>Pie</Card.Footer>
</Card>
```

Form Compound Component:

```
<Form onSubmit={handleSubmit}>
  <Form.Field>
    <Form.Label>Email</Form.Label>
    <Form.Input
      type="email"
      name="email"
    />
    <Form.Error>Error message</Form.Error>
  </Form.Field>
  <Form.Actions>
    <Form.Submit>Guardar</Form.Submit>
  </Form.Actions>
</Form>
```

3. Panel de Configuración

- Sección de tema con preview en tiempo real
- Sección de tipografía con slider o botones
- Sección de densidad con preview
- Sección de notificaciones con toggles
- Botón de resetear a valores por defecto

4. Página Principal

- Mostrar datos del dominio asignado
- Cards que respetan la configuración
- Demostrar que los cambios se aplican globalmente

Estructura del Proyecto

```
3-proyecto/
├── README.md
├── starter/
│   ├── contexts/
│   │   └── ConfigContext.tsx
│   ├── components/
│   │   ├── Card/
│   │   │   └── Card.tsx
│   │   ├── Form/
│   │   │   └── Form.tsx
│   │   └── SettingsPanel/
```

```

├── SettingsPanel.tsx
├── Layout/
│   └── Layout.tsx
├── hooks/
│   └── useLocalStorage.ts
├── App.tsx
└── solution/
    └── [misma estructura con implementación completa]

```

✓ Criterios de Evaluación

Criterio	Puntos
ConfigContext con persistencia	15
Card Compound Component completo	15
Form Compound Component funcional	15
Panel de configuración con todas las opciones	15
Aplicación de estilos según configuración	15
Código limpio, tipado y documentado	10
Adaptación coherente al dominio	15
Total	100

💡 Hints

1. useLocalStorage hook:

```

const useLocalStorage = <T>(key: string, initialValue: T) => {
  // Implementación que sincroniza con localStorage
};

```

2. CSS Variables para temas:

```

[data-theme='dark'] {
  --bg-primary: #1a1a2e;
  --text-primary: #ffffff;
}

```

3. Compound Components con Object.assign:

```

const Card = Object.assign(CardRoot, {
  Header: CardHeader,
  Title: CardTitle,
  Body: CardBody,
  Footer: CardFooter,
});

```

🔗 Recursos

- [React Docs - Context](#)
- [React Patterns - Compound Components](#)
- [CSS Variables](#)